Oh My Zsh: A Practical Guide to Your Shell

Mick Cooney

12 October 2026

Table of Contents

1 Introduction **2**
- 1.1 What This Primer Covers . . . 3
- 1.2 What This Primer Is Not . . . 3
- 1.3 Assumed Background . . . 3

2 Why ZSH Over Bash **3**
- 2.1 Completion . . . 4
- 2.2 Globbing . . . 4
- 2.3 Parameter Expansion . . . 4
- 2.4 History . . . 5
- 2.5 Named Directories . . . 5

3 How Oh My Zsh Works **6**
- 3.1 The `lib/` Layer . . . 6
- 3.2 Enabling and Disabling Plugins . . . 7
- 3.3 Choosing Plugins Without Bloating Startup . . . 7
- 3.4 A `.zshrc` Structure That Scales . . . 7

4 Plugins Worth Knowing Deeply **8**
- 4.1 git . . . 8
- 4.2 fzf . . . 9
- 4.3 zsh-autosuggestions . . . 9
- 4.4 zsh-syntax-highlighting . . . 10
- 4.5 history . . . 11
- 4.6 sudo . . . 11
- 4.7 extract . . . 11
- 4.8 colored-man-pages . . . 12
- 4.9 command-not-found . . . 12
- 4.10 timer . . . 12
- 4.11 rsync . . . 12
- 4.12 systemd . . . 12
- 4.13 mise . . . 13
- 4.14 gh . . . 13

4.15 Commonly Worth Trying Next . . . 13

5 Themes: What a Good Prompt Does **14**
5.1 What Information Belongs in a Prompt . . . 14
5.2 robbyrussell . . . 15
5.3 agnoster . . . 15
5.4 Powerlevel10k (p10k) . . . 15
5.5 Quick Theme Comparison . . . 16

6 Features Most OMZ Users Never Find **16**
6.1 Custom Aliases and Functions in `custom/` . . . 16
6.2 The `omz` Command . . . 17
6.3 Smart Directory Navigation Without `cd` . . . 17
6.4 `z` — Smarter Directory Jumping . . . 17
6.5 ZSH Suffix Aliases . . . 18
6.6 Global Aliases . . . 18
6.7 ZSH Line Editor (ZLE) Widgets . . . 18
6.8 Configuration Areas Worth Periodic Attention . . . 19
6.9 High-Leverage Checks Most People Miss . . . 20

7 Worked Example: Auditing a Real OMZ Setup **21**
7.1 What This Setup Has . . . 21
7.2 What's Missing . . . 21
7.3 Verify Existing Plugins Are Actually Used . . . 22
7.4 Small Additions Worth Considering . . . 22
7.5 Self-Audit Checklist . . . 22

8 Troubleshooting Common Problems **23**
8.1 Shell Startup Feels Slow . . . 23
8.2 Prompt Changes Are Not Taking Effect . . . 23
8.3 Completions Feel Broken or Inconsistent . . . 23
8.4 A Plugin Causes Weird Behaviour . . . 24

9 When OMZ Is Not Enough **24**

10 Quick Reference: What to Do Next **25**

11 Further Reading **25**

---

1 Introduction

Many users first encounter Oh My Zsh through a theme and a familiar plugin such as `git`. This primer looks beyond prompt appearance: it explains what Zsh (the Z shell) provides,

how Oh My Zsh (OMZ) layers plugins and themes over it, and how to decide which parts are useful in your own setup.

It compares Zsh with Bash, explains how OMZ is structured, examines selected plugins and shell features, and ends with a worked configuration audit you can adapt to your setup.

1.1 What This Primer Covers

- What Zsh adds over Bash that matters in daily use
- How Oh My Zsh works under the hood — plugins, themes, the `custom/` directory
- The plugins worth knowing deeply, with real examples of what they do
- Themes: what a good prompt gives you and when to switch
- Zsh features that are easy to overlook
- A concrete worked audit and a checklist you can apply to your own shell

1.2 What This Primer Is Not

This is not a Bash-to-Zsh migration guide, and it does not cover every plugin in the OMZ repository. It does not compare alternative shells or frameworks in depth. The focus is on helping someone who already has OMZ installed understand and configure it.

1.3 Assumed Background

You are comfortable in a terminal, have used Bash or Zsh before, and already have Oh My Zsh installed. You know what a `.zshrc` is. You don't need to know Zsh internals.

As a starting model, Zsh provides programmable completion, filename generation with qualifiers, and a programmable prompt and line editor. Bash also supports programmable completion; the two shells differ in their interfaces and built-in capabilities. OMZ adds configuration, plugins, and themes on top of Zsh rather than replacing the shell.

Verified against primary sources for sections 2–6 and 9; unverified claims are flagged inline.

---

2 Why ZSH Over Bash

Understanding Zsh's own features makes it easier to distinguish them from what OMZ adds. Zsh and Bash are separate shells, and both can be extended with completion functions.

2.1 Completion

Zsh's tab-completion system, called `compsys`, is programmable and context-sensitive. With suitable completion functions loaded, it can offer command-specific flags and arguments, hostnames from Secure Shell (SSH) configuration, container names, systemd units, and manual-page sections. OMZ bundles completion functions for many tools. Bash also has programmable completion, for example through the `bash-completion` project; the available candidates depend on the functions installed and configured in either shell.

For example, a configured completion function can offer Git subcommands after `git <Tab>` or Docker flags after `docker run --<Tab>`. The exact candidates depend on the installed completion definitions and tool version.

2.2 Globbing

Zsh provides glob qualifiers that can filter filename matches by metadata. This can replace some pipelines for tasks that fit the available qualifiers.

```
# Match .log files recursively (**/ is a Zsh glob pattern)
ls **/*.log

# Only files (not directories) modified in the last day
ls *(m-1.)

# Only executable files
ls *(x.)

# Files larger than 1MB
ls *(Lm+1)

# Combine: executable files modified in the last week
ls *(x.m-7)
```

The glob qualifiers (the characters inside `(...)`) filter on file metadata such as type, modification time, size, and permissions. Use them when the selection fits the glob syntax; `find` remains useful for more complex traversal and actions.

2.3 Parameter Expansion

Zsh adds parameter-expansion flags beyond the forms shared with Bash:

```
path="$HOME/projects/example-project/notes.md"

# Bash-compatible: get filename without path
echo ${path##*/}          # notes.md

# ZSH-only: split on delimiter and get last element
echo ${path:t}            # notes.md

# Strip extension
echo ${path:t:r}          # notes
```

```
# Directory part
echo ${path:h}            # your projects/example-project directory

# Uppercase
str="hello world"
echo ${(U)str}            # HELLO WORLD

# Split into array on /
parts=(${(s:/:)path})
echo $parts[1]            # home
```

The `(U)`, `(L)`, `(s:delim:)` and similar parameter flags are Zsh-specific. They can handle some string transformations that might otherwise use external tools such as `awk` or `sed`.

2.4 History

Zsh history behaviour is configurable through options such as:

```
HISTSIZE=100000          # lines in memory
SAVEHIST=100000          # lines on disk
setopt HIST_IGNORE_DUPS  # don't record adjacent duplicates
setopt HIST_IGNORE_SPACE # don't record commands starting with space
setopt SHARE_HISTORY     # share history across sessions in real time
setopt EXTENDED_HISTORY  # record timestamp with each command
```

`SHARE_HISTORY` imports and appends history entries across sessions, so another shell can see commands without being reopened. `HIST_IGNORE_SPACE` omits commands beginning with a space from the shell's history list. Treat it as a convenience, not as protection for secrets: other session logging, shell integrations, or process inspection may still expose a command.

Before copying these into your `.zshrc`, note that OMZ's history library sets `HIST_IGNORE_SPACE`, `HIST_IGNORE_DUPS`, `SHARE_HISTORY`, and `EXTENDED_HISTORY` during initialisation, independently of the selected theme and plugins. The block above is shown for completeness, not because you need to add it yourself.

2.5 Named Directories

Zsh lets you name directories and use them like variables in paths:

```
hash -d work="$HOME/projects"
hash -d proj="$HOME/projects/example-project"

# Now you can use ~work and ~proj anywhere
cd ~proj
ls ~work/another-project
```

Put these in your `.zshrc` and they persist. It's the equivalent of shell aliases for paths, except they work in any context where a path is accepted.

---

3 How Oh My Zsh Works

Understanding the structure of OMZ makes it easier to know what to change and where. The repository lives at `~/.oh-my-zsh/` and has a clear layout:

```
~/.oh-my-zsh/
  lib/           # Core ZSH configuration loaded at startup
  plugins/       # Bundled plugins (git, docker, fzf, etc.)
  themes/        # Bundled themes
  custom/        # Your overrides: themes, plugins, functions
    plugins/     # Third-party or custom plugins go here
    themes/      # Custom themes go here
    aliases.zsh  # Put custom aliases here
    functions.zsh# Put custom functions here (or any .zsh file)
  templates/     # Template for new .zshrc
  tools/         # Install/update scripts
```

When your shell starts, OMZ loads `oh-my-zsh.sh`, which: 1. Loads everything in `lib/` (history config, completion setup, key bindings, utility functions) 2. Sources each enabled plugin's `.plugin.zsh` file 3. Sources everything in `custom/` that ends in `.zsh` 4. Loads the theme

The `custom/` directory is where your personalisations live and where they survive OMZ updates. If you write a custom function and put it in `~/.oh-my-zsh/custom/myconfig.zsh`, it gets loaded automatically. If you put the same function in a bundled plugin file, the next `omz update` will overwrite it.

The theme loading last matters in practice: because it runs after `custom/`, a theme can clobber prompt-related settings you set in a custom file. If a customisation to your prompt doesn't seem to be taking effect, check whether the theme is overwriting it on load.

3.1 The `lib/` Layer

Most users never look at `lib/`, but it does a lot. The files there configure:

- `completion.zsh` — sets up `compsys`, enables menu completion, configures case-insensitive matching
- `history.zsh` — the history options described earlier
- `key-bindings.zsh` — registers Ctrl+R for history search, Ctrl+A/E for line navigation, etc.
- `git.zsh` — the helper functions used by git-aware themes
- `functions.zsh` — utility functions used internally, like `omz_urlencode`

You rarely edit these directly, but knowing they exist explains why certain behaviours appear without you configuring them.

3.2 Enabling and Disabling Plugins

The `plugins` array in `.zshrc` is straightforward:

```
plugins=(
  git
  docker
  fzf
  zsh-autosuggestions
)
```

Each name must match either a directory in `~/.oh-my-zsh/plugins/` or `~/.oh-my-zsh/custom/plugins/`. OMZ looks in both places. Third-party plugins (like `zsh-autosuggestions`) are cloned into `custom/plugins/` and added to the list.

OMZ sources each enabled plugin during shell initialisation, so plugins add startup work. The cost depends on the plugin, configuration, machine, and environment; keep plugins you use and measure before and after changing the list.

3.3 Choosing Plugins Without Bloating Startup

To avoid enabling plugins without a clear use, consider this selection framework:

1. Keep plugins that remove frequent friction from your real workflow (for example `git`, `fzf`, `sudo`, `extract`).
2. Keep plugins that provide completions or aliases for tools you use weekly (`gh`, `docker`, `systemd`, `mise`).
3. Remove plugins that only looked useful once but do not change your daily command lines.

There is no useful universal plugin-count target. A longer list may be fine for your workflow; measure rather than guess:

```
time zsh -i -c exit
```

Run this before and after plugin changes. If startup jumps meaningfully, the extra plugin had a real cost.

3.4 A `.zshrc` Structure That Scales

Many OMZ setups become hard to reason about because `.zshrc` turns into an unsorted script. A predictable layout avoids that:

```
# 1) Environment and path basics
export EDITOR=vim
export PATH="$HOME/bin:$PATH"

# 2) Core OMZ settings
export ZSH="$HOME/.oh-my-zsh"
ZSH_THEME="robbyrussell"
```

```
# 3) Plugin list (must be set before sourcing OMZ)
plugins=(git fzf zsh-autosuggestions)

# 4) Load OMZ
source "$ZSH/oh-my-zsh.sh"

# 5) Local options and bindings
setopt AUTO_CD
setopt CORRECT

# 6) Project/personal customizations
# (aliases/functions in ~/.oh-my-zsh/custom/*.zsh)
```

Two ordering rules matter most in practice:

1. The `plugins=(...)` list must be defined before `source "$ZSH/oh-my-zsh.sh"`.
2. `zsh-syntax-highlighting` must be last in your plugins list.

If something "doesn't load", check these two ordering rules first.

---

4 Plugins Worth Knowing Deeply

This is not a list of every plugin. It is a deep look at the ones that change how you work, with concrete examples of what they actually do.

4.1 git

The `git` plugin ships a large alias set. Two aliases to start with are `gst` (git status) and `gaa` (git add –all). Other examples include:

```
# Commonly used
gst    # git status
ga     # git add
gaa    # git add --all
gcmsg  # git commit -m
gp     # git push
gl     # git pull
gco    # git checkout
gcb    # git checkout -b
gd     # git diff
gds    # git diff --staged
glog   # git log --oneline --decorate --graph

# Less obvious but very useful
gpra   # git pull --rebase --autostash
gcf    # git config --list
gclean # git clean -id (interactive clean)
gwip   # create a work-in-progress commit
gunwip # remove the current WIP commit
```

The `gwip` / `gunwip` pair can help when you need to switch tasks without writing a conventional commit message. `gwip` creates a temporary work-in-progress (WIP) commit; `gunwip` removes it when `HEAD` points to that commit.

The plugin also provides `git_prompt_info()`, which most themes use to show the current branch and status in the prompt.

4.2 fzf

`fzf` is a fuzzy finder for the terminal. When `fzf` and its shell key bindings are installed and enabled, the integration provides these key bindings:

```
Ctrl+R    – fuzzy search your command history
Ctrl+T    – fuzzy find a file in the current directory tree
Alt+C     – fuzzy cd into a subdirectory
```

If Ctrl+R is the only binding you use, try the others. `Ctrl+T` can help when you remember a filename but not its path; `Alt+C` offers another way to navigate nested directories.

`fzf` also integrates with commands directly:

```
# Preview files while browsing
fzf --preview 'cat {}'

# Select one of *your* processes, then send the normal TERM signal.
# Cancel or an empty selection does nothing. Review the command before running it.
selected_pid="$(
  ps -u "$USER" -o pid=,command= |
    fzf --prompt='process> ' |
    awk '{print $1}'
)"
[[ -n "$selected_pid" ]] && kill -- "$selected_pid"

# Open a selected local branch; canceling leaves the current branch unchanged.
selected_branch="$(git branch --format='%(refname:short)' | fzf --prompt='branch> ')"
[[ -n "$selected_branch" ]] && git switch -- "$selected_branch"

# Put a selected history entry on the command line for review; it does not run yet.
selected_command="$(history | fzf --prompt='history> ' | awk '{$1=""; sub(/^ /, ""); print}')"
[[ -n "$selected_command" ]] && print -z -- "$selected_command"
```

The underlying tool accepts any input on stdin and returns the selected line on stdout, so it composes with everything. The OMZ plugin just provides the shell integration; learning `fzf`'s flags unlocks the full power. Treat a process picker as a destructive operation: inspect the selected command, start with the normal `TERM` signal, and reserve `kill -9` for a process that has not responded. Do not use `eval` to re-run fuzzy-selected history: loading the command with `print -z` gives you a final chance to inspect or edit it before pressing Enter.

4.3 zsh-autosuggestions

This plugin shows a greyed-out suggestion as you type, based on your history. The plugin's model splits into two groups of key bindings: widgets that accept the *whole* suggestion, and

widgets that accept it one word at a time.

```
# You type:
git commit

# It shows (in grey):
git commit -m "fix typo in README"   ← suggestion from history

# Right arrow, End, and Ctrl+F all accept the whole suggestion
# Alt+F accepts one word at a time
```

By default, right arrow (`forward-char`), `End` (`end-of-line`), and Ctrl+F (also bound to `forward-char` in emacs mode) are all in `ZSH_AUTOSUGGEST_ACCEPT_WIDGETS` — that array controls which movement widgets, when invoked, accept the entire suggestion rather than just moving the cursor. There's no character-at-a-time acceptance by default; anything in that array takes the whole line.

Word-by-word acceptance is a separate array, `ZSH_AUTOSUGGEST_PARTIAL_ACCEPT_WIDGETS`, and it's driven by `forward-word` — Alt+F in emacs mode. There's no default Ctrl+Space binding for this.

If you want a custom key to accept a suggestion (whole or partial), you don't invent a new widget name — you bind an existing movement widget and add it to the relevant array. For example, to make Ctrl+Space accept one word at a time:

```
bindkey '^ ' forward-word
ZSH_AUTOSUGGEST_PARTIAL_ACCEPT_WIDGETS+=(forward-word)
```

One tuning option worth knowing: by default it only suggests from history. You can also enable completion-based suggestions:

```
ZSH_AUTOSUGGEST_STRATEGY=(history completion)
```

This means that if history has no match, it falls back to Zsh completions.

4.4 zsh-syntax-highlighting

This plugin is not in the default OMZ bundle — it requires a separate install. It is, however, one of the highest-value things you can add. It highlights commands in real time as you type:

- Commands that exist: green
- Commands that don't exist: red
- Strings and quoted arguments: yellow

Options/flags and pipes/redirects/semicolons are recognised as distinct token types internally, but the default `main` highlighter styles leave them unstyled — no colour unless you configure `ZSH_HIGHLIGHT_STYLES` yourself. If you want flags or separators to stand out visually, that's a `ZSH_HIGHLIGHT_STYLES` customisation, not default behaviour.

The value is catching typos before you press Enter. You type `pythno script.py` and the red highlight catches it immediately. You type `ls --recusrive` and it's red before you hit Enter. After using it for a week you will miss it in any shell that doesn't have it.

Install:

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git \
  ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
```

Then add `zsh-syntax-highlighting` to your `plugins` array. It must be **last in the list** — this is important; the plugin works by modifying the line buffer, and it needs to run after everything else.

4.5 history

The `history` plugin provides convenient aliases:

```
h          # full history (with timestamps if HIST_STAMPS is set in .zshrc)
hs         # history | grep (case-sensitive)
hsi        # history | grep (case-insensitive)
# Find all git commands you've run
hs git

# Find recent docker run commands
hsi "docker run"
```

These aliases provide shortcuts for repeated filtered-history searches.

4.6 sudo

Press `Esc Esc` (two taps) to prepend `sudo` to the current command line or re-run the last command with `sudo`.

```
# You run:
apt install something-big
# Permission denied.

# Press Esc Esc
# Line becomes:
sudo apt install something-big
```

4.7 extract

The `extract` plugin provides a single command `x` (or `extract`) that works on any archive format:

```
x archive.tar.gz
x archive.zip
x archive.bz2
x archive.7z
x archive.tar.xz
x archive.rar
```

You never need to remember whether it's `tar -xzf` or `tar -xjf` or `unzip` or `7z x`. The plugin detects the format and uses the right tool. It works as long as the relevant tool is installed on the system.

4.8 colored-man-pages

Makes `man` pages render with colour instead of monochrome. The improvement is significant for readability, especially for long man pages with lots of structure. No configuration needed; just include it in your plugins list.

4.9 command-not-found

When you type a command that doesn't exist, instead of a bare "command not found" error, the shell suggests the package you need to install:

```
$ tree
zsh: command not found: tree

Command 'tree' not found, but can be installed with:
sudo apt install tree
```

Requires a compatible system `command-not-found` handler to be installed and configured; support differs by platform. See the plugin's supported-platform list before relying on it.

4.10 timer

Shows how long each command took, appended to the right side of your prompt after the command completes. Useful for scripts and builds where you want a rough sense of runtime without explicitly using `time`.

```
some-long-command           [12s]
```

No configuration needed. If you find it noisy for short commands, set a minimum threshold:

```
TIMER_THRESHOLD=2  # only show if command took > 2 seconds
TIMER_FORMAT='%d seconds'
```

4.11 rsync

Provides aliases for common `rsync` patterns:

```
rsync-copy      # rsync --partial --progress -h --archive (archive, progress, human-readable)
rsync-move      # rsync --partial --progress -h --remove-source-files --recursive
rsync-update    # rsync --partial --progress -h --update --archive (skip files newer at destination)
rsync-synchronize # rsync --partial --progress -h --update --archive --delete (full sync)
```

The flags are ones you'd reach for often but never quite remember the exact combination of.

4.12 systemd

Provides short aliases for common `systemctl` operations. There's no `journalctl` coverage in this plugin — if you want short aliases for log commands, those are worth adding yourself.

```
sc-status      # sudo systemctl status
sc-start       # sudo systemctl start
sc-stop        # sudo systemctl stop
sc-restart     # sudo systemctl restart
sc-enable      # sudo systemctl enable
sc-enable-now  # sudo systemctl enable --now
```

```
sc-disable     # sudo systemctl disable
sc-disable-now # sudo systemctl disable --now
sc-mask-now    # sudo systemctl mask --now
sc-list-units  # sudo systemctl list-units
```

The privileged aliases all embed `sudo` for you, so you don't type it separately. Every one of them also has a `scu-` twin that drops the `sudo` and adds `--user`, for managing your own user-level units instead of system-wide ones — `scu-status` is `systemctl --user status`, `scu-restart` is `systemctl --user restart`, and so on. If you run anything as a systemd user service, the `scu-` family is arguably more useful day to day than the system-wide aliases.

If you interact with systemd services regularly, these cut down the typing considerably.

4.13 mise

The `mise` plugin initialises `mise` (the polyglot version manager, formerly `rtx`) and enables its completions. It runs `mise activate zsh` during shell startup so that tool versions are correctly set in every shell session. Without this, you'd need to call `eval "$(mise activate zsh)"` manually in your `.zshrc`. That `eval` is appropriate only for initialisation code emitted by a locally installed, trusted executable; never adapt the pattern to execute output fetched from the network.

4.14 gh

Completions for the GitHub CLI (`gh`). The plugin runs `gh completion --shell zsh` and caches the result, so you get tab-completion for `gh`'s commands and flags. It doesn't dynamically complete repository names or PR/issue numbers — that would need live API calls, which this plugin doesn't make. If you use `gh` frequently, the command/flag completion alone saves a lot of flag-hunting.

4.15 Commonly Worth Trying Next

These are common upgrades in mature setups. None are mandatory, but each can be high leverage in the right workflow.

1. **`vi-mode`**

- Gives Vim-style command-line editing in Zsh's line editor.
- Best if you already use Vim/Neovim motions.
- Skip if you strongly prefer emacs bindings.

2. **`direnv` integration**

- Auto-loads per-project environment via `.envrc`.
- Useful for separating project-specific executable search paths and environment variables.
- Requires the `direnv` binary and disciplined `.envrc` review.

3. **`kubectl` and cloud CLI plugins (`aws`, `gcloud`, `azure`)**

- Useful completion and aliases when those CLIs are daily tools.
- Not worth loading if you use them only occasionally.

4. **One version manager strategy (`mise` or `asdf`)**

- For polyglot stacks, one manager lowers cognitive overhead.
- Avoid stacking multiple managers unless compatibility forces it.

5. **`starship` prompt (theme alternative)**

- Strong choice for cross-shell consistency with TOML config.
- Worth trialling if you routinely jump between shells.

---

5 Themes: What a Good Prompt Does

The theme controls what your prompt looks like. When choosing one, consider which information helps you act and whether the prompt remains readable and responsive in your terminal.

5.1 What Information Belongs in a Prompt

A practical prompt should show:

1. **Current directory** — always. Abbreviated to the last 2-3 components if deep.
2. **Git state** — current branch, whether there are staged changes, unstaged changes, untracked files, and whether the branch is ahead/behind the remote.
3. **Last command exit code** — some indication of whether the last command succeeded or failed.
4. **Optionally:** active virtual environment, tool versions, execution time, user@host for remote sessions.

What a prompt should not do: show things that never change (always the same user on the same machine), show timestamps on every line (the `timer` plugin handles that better), or take noticeable time to render.

5.2 robbyrussell

The default theme. Clean, minimal:

```
➜  example-project git:(main) ✗
```

Shows directory and git branch. The ✗ indicates there are uncommitted changes. It's good for its simplicity, but it doesn't show whether changes are staged vs unstaged, doesn't show remote divergence, and gives no indication of exit status.

5.3 agnoster

A powerline-style theme that uses special Unicode characters to create a segmented prompt:

```
you@hostname  ~/projects/example-project  main ✘
```

(The segments have coloured backgrounds in the actual terminal.) Shows user, host, path, git branch, and git state. More information than robbyrussell. Requires a Nerd Font or Powerline-patched font; without one, the segment characters render as boxes.

A common `agnoster` variant, `agnoster-timestamp-newline`, adds a timestamp and puts command input on its own line, which makes long paths more readable.

5.4 Powerlevel10k (p10k)

Powerlevel10k is a third-party theme, not bundled with OMZ. As of 12 October 2026, its upstream README warns of very limited support: no new features are planned, most bugs will remain unfixed, and help requests will be ignored. An existing setup can still work well, but do not assume that future compatibility problems will be fixed. For a new setup, compare its features with Starship and Pure before accepting that maintenance constraint.

What makes p10k different:

- **Configuration wizard** — runs `p10k configure` and walks you through every prompt option interactively, showing previews. You end up with a `.p10k.zsh` config that captures exactly what you chose.
- **Instant prompt** — p10k renders a partial prompt immediately, before the shell finishes loading. Shell startup feels instant even with slow plugins.
- **Async segments** — git status and other slow segments render asynchronously so they don't block input.
- **Transient prompt** — old prompts in the scroll buffer collapse to a minimal form, keeping the terminal cleaner.

For a full git-aware, multi-context prompt (git state, virtualenv, node version, exit status, command timing), p10k's instant prompt and configuration wizard remain useful features. Choose it if those features justify the limited support for your setup.

Install:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git \
  ${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

Then set `ZSH_THEME="powerlevel10k/powerlevel10k"` and run `p10k configure`.

5.5 Quick Theme Comparison

If you are deciding between common options, this is the short version:

Theme	Strength	Tradeoff	Best For
`robbyrussell`	Minimal, no setup	Limited status detail	Simple prompt, low cognitive load
`agnoster`	Denser git/path signal	Needs patched font, can feel busy	Users who want always-visible context
`powerlevel10k`	Rich status, fast rendering, wizard	Very limited upstream support; extra config	Users who accept the maintenance constraint
`pure`	Clean async prompt	Less built-in segment breadth	People who want modern and minimal
`starship`	Cross-shell consistency	External binary and separate config	Mixed-shell workflows

---

6 Features Most OMZ Users Never Find

6.1 Custom Aliases and Functions in `custom/`

Any `.zsh` file in `~/.oh-my-zsh/custom/` is sourced automatically. This means you don't need to clutter `.zshrc` with personal aliases — put them in `~/.oh-my-zsh/custom/aliases.zsh` and they load automatically and survive OMZ updates:

```
# ~/.oh-my-zsh/custom/aliases.zsh

alias ll='ls -lAh --color=auto'
alias ports='ss -tlnp'
# This contacts a third party, which sees your public IP address and request metadata.
# Do not use it from a network where that disclosure is sensitive.
alias myip='curl --fail --silent --show-error https://api.ipify.org'
alias reload='source ~/.zshrc'
alias ...='cd ../..'
alias ....='cd ../../..'
```

Functions go in the same place or their own file:

```
# ~/.oh-my-zsh/custom/functions.zsh

# Create directory and cd into it
mcd() { mkdir -p "$1" && cd "$1" }

# Quick find by name
f() { find . -name "*$1*" 2>/dev/null }

# Show the last N lines of a log file with color
lf() { tail -f "${1:-/var/log/syslog}" | grep --color=auto -E "ERROR|WARN|INFO|$" }
```

6.2 The `omz` Command

OMZ ships a management command that most users don't know exists:

```
omz update              # Update oh-my-zsh
omz plugin list         # List all available plugins
omz plugin enable git   # Enable a plugin without hand-editing .zshrc
omz plugin disable git  # Disable a plugin
omz theme list          # List available themes
omz theme use agnoster  # Switch theme temporarily
omz changelog           # Show recent OMZ changes
```

The `plugin list` output is useful for discovery — there are plugins for tools you use that you may not know OMZ ships.

6.3 Smart Directory Navigation Without `cd`

Zsh can change directories without typing `cd`, if you enable it:

```
setopt AUTO_CD
```

With this set, typing a directory name alone (without `cd`) changes into it:

```
# Without AUTO_CD:
cd ~/projects/example-project

# With AUTO_CD:
~/projects/example-project
# or if it's in CDPATH:
example-project
```

Combine with `CDPATH` to jump to frequently used directories:

```
CDPATH=.:~:~/projects
```

With this set, typing `example-project` from anywhere searches `.`, `~`, and `~/projects` for that directory name and changes into it if found.

6.4 `z` — Smarter Directory Jumping

The `z` plugin (bundled with OMZ) tracks visited directories and lets you jump to matches by partial name. Its ranking combines visit frequency with time since the last visit:

```
# After you've visited ~/projects/example-project a few times:
z example       # cd's to ~/projects/example-project
z project       # same
```

```
# It picks the highest-ranked match for the pattern
z notes         # might jump to ~/projects/example-project/notes
```

This combined ranking (often called "frecency") uses both visit count and access recency. It can reduce repeated path entry in workflows where you revisit the same directories.

`zoxide` is a separate directory-jumping tool with a similar use. It integrates with Zsh and can be configured to replace `cd`. As with other shell initialisation snippets, only `eval` output from a locally installed executable you trust:

```
# Install zoxide, then add to .zshrc:
eval "$(zoxide init zsh --cmd cd)"
# Now 'cd' uses zoxide's frecency ranking
```

6.5 ZSH Suffix Aliases

Zsh supports suffix aliases — aliases that apply to the file extension, so the shell knows what program to use to open a file type:

```
alias -s py=python3
alias -s md=glow       # terminal markdown renderer
alias -s pdf=evince
alias -s html=firefox
```

With these, typing `script.py` at the prompt opens it with `python3`, `README.md` opens with `glow`, and so on. You don't need to prefix the program name.

6.6 Global Aliases

Regular aliases only work at the start of a command. Global aliases work anywhere in a command line:

```
alias -g G='| grep'
alias -g L='| less'
alias -g T='| tail -f'
alias -g NE='2>/dev/null'
alias -g NUL='>/dev/null 2>&1'
# With global aliases:
ps aux G python     # ps aux | grep python
cat big_log T       # cat big_log | tail -f
some-command NUL    # some-command >/dev/null 2>&1
```

This is a Zsh-specific feature with no Bash equivalent.

6.7 ZSH Line Editor (ZLE) Widgets

Zsh's line editor is programmable. You can write functions and bind them to key sequences:

```
# Paste last argument of last command (already bound to Alt+. by default)
# But you can write custom widgets:

# Widget: prepend sudo to the current command
prepend-sudo() {
  BUFFER="sudo $BUFFER"
  CURSOR=$((CURSOR + 5))
}
```

```
zle -N prepend-sudo
bindkey '^[s' prepend-sudo   # Alt+S
```

Realise halfway through typing a command that it needs root? Alt+S prepends `sudo` without you having to jump to the start of the line, type it, and jump back. `BUFFER` is the whole command line as a string, and `CURSOR` is the cursor's character offset into it — direct edits to both are the basic pattern for any custom ZLE widget.

Stock Zsh (not something OMZ adds) provides a related widget: `push-line` (bound to Ctrl+Q in the emacs keymap by default). If you're mid-command and realise you need to run something else first, Ctrl+Q parks your current input, you run what you need, and your original command returns at the next prompt. If terminal flow control is enabled (`stty ixon`), the terminal driver may intercept Ctrl+Q as XON (the resume-output control character) before Zsh receives it. If so, run `stty -ixon` or use the Esc-q alternative binding.

6.8 Configuration Areas Worth Periodic Attention

Beyond plugins and themes, these settings often determine whether a shell feels excellent or annoying:

1. **History policy**

- Validate `HISTSIZE`, `SAVEHIST`, and history sharing behaviour against real usage.
- Keep `HIST_IGNORE_SPACE` active and use leading-space deliberately for sensitive commands.

2. **Completion behaviour and cache health**

- Slow completion is often stale cache or plugin overload.
- Revisit completion behaviour after major toolchain changes.

3. **Editing mode and keymap consistency**

- Pick `emacs` or `vi` intentionally and stay consistent.
- If using vi mode, make insert/normal status obvious in the prompt.

4. **Prompt signal-to-noise**

- Keep status fields that change decisions (git state, exit code, env) and remove decorative noise.
- Re-check latency whenever adding new prompt segments.

5. **Executable search path (`PATH`) ordering and shim collisions**

- Ensure one intentional winner when multiple version managers provide the same binary.

- Remove duplicate additions to the executable search path across shell files.

6. **Custom file hygiene**

- Keep aliases/functions/options organised in `custom/` files instead of growing one giant `.zshrc`.
- Periodically delete stale helpers you no longer use.

7. **Terminal/font compatibility**

- Nerd/Powerline prompts require compatible fonts.
- If symbols render as boxes, fix terminal/font config before debugging shell logic.

6.9 High-Leverage Checks Most People Miss

These are small checks that often catch real issues quickly:

1. **Custom guard files are loaded at all**

- If you keep policy helpers in standalone files (for example, host-execution guards), verify they are explicitly sourced.
- A correct file in the wrong location is still inactive.

2. **Duplicate settings agree across files**

- If key behaviour is configured in both plugins and aliases/functions (for example SSH key lifetime), align them.
- Inconsistent values create hard-to-explain behaviour after reloads.

3. **Generated completions are not rebuilt every shell start**

- Prefer one-time generation for tools like `just` unless you truly need dynamic regeneration.
- Rebuilding completion scripts on every startup adds avoidable latency and extra disk I/O.

4. **Theme and prompt ownership are not mixed accidentally**

- If you use an OMZ theme, avoid silently overriding `PROMPT` later unless intentional.
- Mixed ownership causes confusing prompt changes and harder troubleshooting.

---

7 Worked Example: Auditing a Real OMZ Setup

This is an audit of one representative configuration. Read it as an example process you can adapt, not as a recommendation for every OMZ setup.

7.1 What This Setup Has

Active plugins in this example:

```
git  ssh-agent  colored-man-pages  command-not-found  extract  history
docker  timer  sudo  gh  podman  systemd  chezmoi  rsync  fzf
mise  zsh-autosuggestions  zsh-claude-code-shell
```

The list reflects this user's toolchain. The value of `timer`, `sudo`, `extract`, `history`, `colored-man-pages`, `fzf`, and `zsh-autosuggestions` depends on which commands and interactions the user relies on. The tool-specific plugins (`docker`, `podman`, `systemd`, `gh`, `chezmoi`, `rsync`, `mise`) match tools in this example environment.

The history configuration retains up to 100,000 entries and shares history across sessions. Choose `HISTSIZE` and `SAVEHIST` for your storage and search needs; a larger history can also make searching less focused.

7.2 What's Missing

`zsh-syntax-highlighting` — an optional addition.

This third-party addition applies styles to command-line tokens as you type. The default styles and recognised syntax do not guarantee that a command exists or will succeed. Try it if this feedback suits your workflow.

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git \
  ~/.oh-my-zsh/custom/plugins/zsh-syntax-highlighting
```

Add `zsh-syntax-highlighting` to the plugins list, and put it **last**:

```
plugins=(
  ...your existing plugins...
  zsh-syntax-highlighting   # must be last
)
```

`z` for directory jumping.

If you revisit directories, the `z` plugin may reduce repeated path entry. It is bundled with OMZ; add it to the plugins list and let its database collect visits before trying queries such as `z research` or `z building`:

```
plugins=(... z ...)
```

Consider the active theme choice.

This example has `agnoster-timestamp-newline` available but uses `robbyrussell`. That may be intentional; compare the layouts against your terminal width and the information you want to see. p10k's configuration wizard lets you select prompt elements interactively.

7.3 Verify Existing Plugins Are Actually Used

Before adding another plugin, exercise the high-value ones already installed in this setup:

- `fzf`: use `Ctrl+T` for file arguments and `Alt+C` for directory jumping, not only `Ctrl+R`.
- `sudo`: press `Esc Esc` after a failed command to put `sudo` in front of it.
- `extract`: use `x archive.tar.gz` instead of memorising archive-specific flags.
- `history`: use `hs docker` instead of typing `history | grep docker`.

The earlier plugin sections explain these capabilities in detail; this audit is checking that they have become habits, not recommending more plugins.

7.4 Small Additions Worth Considering

A few things that are low-effort and pay off quickly:

```
# In ~/.oh-my-zsh/custom/aliases.zsh

# Directory navigation shortcuts
alias ..='cd ..'
alias ...='cd ../..'
alias ....='cd ../../..'

# Quick list
alias ll='ls -lAh --color=auto'

# Git shortcuts beyond what the git plugin provides
alias gst='git status'   # already from git plugin, but worth knowing

# Show port usage
alias ports='ss -tlnp'
```

And a few Zsh options worth adding to your `.zshrc` if they're not there (note: `HIST_IGNORE_SPACE` and friends from the History section earlier are already set by OMZ's `lib/history.zsh` — no need to add those yourself):

```
setopt AUTO_CD          # type a directory name to cd into it
setopt CORRECT          # suggest corrections for mistyped commands
```

7.5 Self-Audit Checklist

Use this checklist against your own shell every few months:

1. Does every enabled plugin remove regular friction, or is it leftover experimentation?
2. Is startup still responsive (`time zsh -i -c exit`) after a plugin or theme change?
3. Are high-impact basics present (`fzf`, `zsh-autosuggestions`, `zsh-syntax-highlighting`, `z`)?
4. Is your prompt giving useful state (branch, dirty state, exit code) without visual noise?

5. Are aliases/functions in `custom/` so updates do not overwrite your work?
6. Is executable search path order intentional, with no duplicate manager shims?
7. Do key bindings match your editing mode (`emacs` or `vi`) without surprises?
8. Are all critical custom files actually sourced (not just present on disk)?
9. Do duplicated settings (like SSH key lifetimes) match across aliases, plugins, and helper scripts?
10. Are tool completions generated once instead of re-generated on every shell startup?

Several "no" answers can identify configuration areas worth reviewing; the checklist is a prompt for investigation, not a score or diagnosis.

---

8 Troubleshooting Common Problems

8.1 Shell Startup Feels Slow

Measure first:

```
time zsh -i -c exit
```

Then isolate cause by temporarily removing half the plugin list, re-measuring, and repeating (binary search). Typical culprits are plugin initializers that shell out to external tools at startup.

8.2 Prompt Changes Are Not Taking Effect

Check order:

1. Is `ZSH_THEME` set before sourcing OMZ?
2. Is another file in `custom/` modifying `PROMPT` after theme load?
3. If using p10k, is `~/.p10k.zsh` being sourced as expected?

Because theme loading happens late in OMZ startup, prompt variables changed earlier can be overwritten.

8.3 Completions Feel Broken or Inconsistent

Quick checks:

1. Confirm the relevant plugin is in `plugins=(...)`.
2. Start a clean shell and test again.

3. Update OMZ (`omz update`) if the installed completion files may be out of date.

If completion for one command is still missing, inspect that plugin's `.plugin.zsh` file and verify the completion function is actually loaded.

8.4 A Plugin Causes Weird Behaviour

Run a temporary minimal profile:

```
plugins=(git)
```

If the issue disappears, re-add plugins in small groups until it returns. This narrows the search before you inspect each plugin's source.

---

9 When OMZ Is Not Enough

OMZ is one way to organise Zsh configuration. Whether it suits you depends on the features, startup behaviour, and maintenance model you want.

Startup speed. OMZ initialises its configuration, completions, plugins, and theme when a shell starts. The time depends on the machine, configuration, cache state, and plugins, so measure your own interactive startup (for example, with `time zsh -i -c exit`) before changing frameworks. Other plugin managers offer different loading and configuration models; compare them using the same workload rather than assuming they will be faster.

Configuration control. OMZ supplies defaults for completion, history, and key bindings. If you want to manage those pieces yourself, a minimal Zsh configuration or another plugin manager may offer a better fit, at the cost of more setup and maintenance.

Alternatives worth comparing. Prezto is another Zsh configuration framework. Fish has its own interactive features and scripting syntax, so Bash and Zsh scripts do not transfer unchanged. Nushell uses structured data in its pipeline model rather than treating command output only as text. Compare the features and migration cost against your own use; these differences do not establish a universal ranking.

If you want a preconfigured Zsh setup, OMZ with a deliberately chosen plugin list is one option. A theme such as p10k adds its own maintenance trade-offs, described above.

---

10 Quick Reference: What to Do Next

In order of impact:

1. **Install `zsh-syntax-highlighting`** — see the install command in the plugin section above. Add it last in your plugins list.
2. **Add `z`** to your plugins list (no install needed, it's bundled). Use it for a week.
3. **Try `Ctrl+T` and `Alt+C`** with `fzf` — fuzzy file and directory completion. They're already wired up.
4. **Internalise `Esc Esc`** (sudo plugin) and `x` (extract plugin) — you have both, just build the habit.
5. **Consider p10k** if you want a richer prompt with git state detail.
6. **Read `omz plugin list`** — there may be completions for tools you use daily that you didn't know OMZ ships.

---

11 Further Reading

- **Oh My Zsh plugin documentation** — browse bundled plugin descriptions, then inspect the linked README or source for the installed version because behaviour can change.
- **Oh My Zsh customisation guide** — explains custom plugins, themes, and override files.
- **Zsh manual: Expansion, Options, Line Editor, and Completion System** — consult the sections on parameter expansion, filename generation, history options, widgets, and command completion; local installations also provide `man zsh`.
- **zsh-autosuggestions** — documents the suggestion strategies, widgets, and key-binding arrays used in this primer.
- **zsh-syntax-highlighting installation** — documents installation and why it must be sourced after other plugins.
- **fzf shell integration** — documents its default command-line key bindings and their installation.
- **Oh My Zsh `z` plugin** — documents the bundled directory-jumping plugin and its matching behaviour.

- **Oh My Zsh `command-not-found` plugin** — lists supported platform handlers and setup details.
- **Powerlevel10k README** — read its support notice and feature documentation before adopting it; its current support status is summarised above.
- **zoxide documentation** — explains its Zsh initialisation and directory-jumping options.
- **Zsh mailing list** — a place for questions about Zsh itself; check the shell and framework versions when applying advice.